

Homework — Agentic loop: implementing BayesFactor

Purpose

Practice the **agentic coding loop** from the lecture: run a Gemini-2.5-powered test-fix cycle against a real project, inspect what the model produces, and reflect on how you managed the interaction.

Background

In The AI-coding landscape we saw this pattern:

`generate code → run tests → read errors → edit code → run tests again`

The script `files/agent_loop/run_agent_loop.py` implements exactly this loop for any Python project with a `unittest` test suite. You give it a task description and a project directory; it calls Gemini, writes updated files, runs the tests, and repeats until they pass or the iteration cap is reached.

This homework has two parts: a warm-up to verify your setup, and a main task that applies the same loop to the `BayesFactor` class from Module 04.

Setup

You need:

- The class Docker container running.*
- A Gemini key in a JSON file (see `files/agent_loop/README.md`), ideally under `/workspace/secrets/`.
- Your Module 04 `BayesFactor` test suite.

* Seriously, do *not* let a running agent out of its cage!

Part 1 — Warm-up: Fisher z-transform

This part verifies your setup and lets you see the loop succeed on a small, self-contained example — the same function shown in the lecture.

Steps:

1. Copy `files/agent_loop/warmup/` into a convenient working directory in the container (keep the folder structure).
2. Confirm the tests fail before the loop starts:

```
python -m unittest discover -s tests
```


You should see `NotImplementedError`.
3. Write a one-sentence `task.txt` in the warmup directory:

Implement `fisher_z_transform` in `fisher.py` so all tests pass. Do not modify the test file.

4. Run the agent loop (from the repo root, or adjust the path):

```
python files/agent_loop/run_agent_loop.py \
  --project warmup/ \
  --task-file warmup/task.txt \
  --model gemini-2.5-flash-lite \
  --max-iters 3
```

5. Confirm tests pass. Keep the terminal output — you will reference it later.
-

Part 2 — Main task: BayesFactor

Steps:

1. In your Module 04 BayesFactor repo, create a new branch:

```
git switch -c agent-loop-hw
```

2. Copy `files/agent_loop/bayes_factor_stub.py` over your existing implementation:

```
cp files/agent_loop/bayes_factor_stub.py bayes_factor.py
```

3. Confirm the tests now fail:

```
python -m unittest discover -s tests
```

4. Write a `task.txt` in your BayesFactor repo root. Your task description must include:

- The class name (`BayesFactor`) and all required method signatures (`__init__(n, k)`, `likelihood(theta)`, `evidence_slab()`, `evidence_spike()`, `bayes_factor()`).
- The prior specifications: $\text{slab} = U(0, 1)$, $\text{spike} = U(a, b)$ with the values of a and b from Etz et al. (2018). (Hint: they are *not* 0.4999 and 0.5001!)
- The instruction not to modify the test file.
- The instruction not to change the constructor signature.

5. Before you run the loop, think about the possibility that the agent will try to modify the test file. How could it do that? How will you stop it?

6. Run the loop with a 3-iteration cap:

```
python files/agent_loop/run_agent_loop.py \
  --project . \
  --task-file task.txt \
  --model gemini-2.5-flash-lite \
  --max-iters 3
```

7. Review the implementation the model produced. Read through every method before accepting it. How does it smell?
-

Deliverables

Submit the following in your Module 04 BayesFactor repository (on the `agent-loop-hw` branch):

`task.txt`

`agent_loop_output.txt` # terminal output of the loop run, copied to a file

And in a file called `reflection.md`:

A reflection of **250 words** describing how you actually did this: how you set up the loop, what you tried, what happened, did you have to intervene?

Do not commit your API key file!

Assignment constraints

- ☐ `task.txt` states the class name, all method signatures, prior values, and the no-test-modification rule.
- ☐ `--max-iters 3` was the cap used.
- ☐ The test file was not modified.
- ☐ The API key file was not committed.
- ☐ `reflection.md` is 250 words.

Reference

- The AI-coding landscape — lecture background
- `files/agent_loop/README.md` — full script documentation and flags
- Module 04 homework — BayesFactor specification and prior values